Reviewer Pack — Minimal Symmetric Difference Divisors of Boolean Functions o…

Entry 1dec158e845e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 05:01:22 UTC

Minimal Symmetric Difference Divisors of Boolean Functions over AC⁰ Circuit Width

Entry ID: `1dec158e845e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 05:01:22 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Complexity Theory: AC⁰ Circuit Width

Statement:

> ['For any boolean function f computable by an AC⁰ circuit C with n inputs and size s, the smallest number of distinct symmetric difference divisors of f over all possible boolean functions of size at most s is at least Θ(log^2(n)/s).', 'Equivalently, for any boolean function f computable by an AC⁰ circuit C with n inputs and size s, the symmetric difference divisor degree of f (i.e., the number of distinct boolean functions g such that g XOR f is constant) satisfies D(f) ≥ Θ(log^2(n)/s).', 'If there exists a boolean function f computable by an AC⁰ circuit C with n inputs and size s for which D(f) < Θ(log^2(n)/s), then this conjecture is false.']

Rationale (proposer's reasoning):

> ['Algebraic combinatorics provides tools to study the structure of functions through their divisors. A connection between the divisor theory and AC⁰ circuit width could potentially expose a new avenue for proving lower bounds on PARITY.', 'The symmetric difference divisor degree has not been extensively studied in complexity theory, but it is related to the algebraic structure of boolean functions. Understanding this relationship might lead to stronger lower bounds on circuit complexity.', 'This conjecture focuses on the divisor count, which is a natural quantity to consider for capturing the complexity of a function.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e06d73d6eeae682a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> To test the conjecture, generate random AC⁰ circuits of size s up to 40 inputs, compute their symmetric difference divisors, and measure D(f). A result supports the conjecture if

for all seeds, the ratio D(f)/s is greater than or equal to Θ(log^2(n)/s) for all f. It falsifies the conjecture if there exists any seed where for at least one f, the ratio D(f)/s is less than Θ(log^2(n)/s).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.2s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n, s):
        # Simplified AC⁰ circuit generation (not actual AC⁰)
        return [random.choice([0, 1]) for _ in range(s)]

    def symmetric_difference(f, g):
        return [x ^ y for x, y in zip(f, g)]

    def is_constant(lst):
        return all(x == lst[0] for x in lst)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Test with 5 random AC⁰ circuits of size s up to n inputs
            s = random.randint(1, n)
            circuit = generate_ac0_circuit(n, s)
            metric_value = 0

            for g in itertools.product([0, 1], repeat=n):
                if is_constant(symmetric_difference(circuit, g)):
```

```
                    metric_value += 1

            total_metric_value += metric_value
            instances_tested += n

    mean_metric_value = Fraction(total_metric_value, instances_tested)
    conjecture_holds = all(mean_metric_value >= (math.log(n) ** 2 / s) for n in n_values for s in

    return {
        "metric_name": "symmetric_difference_divisor_degree",
        "metric_value": float(mean_metric_value),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within a reasonable timeframe.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12581
2	preregistration	ollama_remote	glm4:latest	0	0	6670
3	novelty	ollama_remote	glm4:latest	0	0	5996
4	novelty	ollama_remote	glm4:latest	0	0	5115
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13775
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8597
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9821
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7794
9	judge	ollama_remote	glm4:latest	0	0	8334

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 78680 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1dec158e845e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1dec158e845e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1dec158e845e.tar.gz` (if generated)